M4 Pre-Event B decision

Group 35

Concrete Design Problem

The original study reuses one dependency cache across all JDK builds of a project, so an earlier build can influence a later one. The cache is only cleared once a project's builds on every JDK version are finished.

Item	Detail
Flaw	Shared dependency cache across JDK 11, 17, 21 and 23 builds of the same project
Where found	Original authors' `execution/run-builds.py`
Risk	Build results are not fully independent per JDK version
Our decision	No change; kept the original approach for comparability

Evidence from run-builds.py

Four parts of the original authors' script show the cache is shared across JDK versions and only cleared between projects. Line numbers refer to commit 16a4b3d.

Lines	What the code does	Why it matters
14–16	`CACHE_DIRS` lists the Gradle, Maven and Ivy dependency/wrapper caches	These are the folders that persist between builds
46–49	`get_volume_name()` builds the Docker volume name from the image name and cache path only	The JDK version is not part of the name, so every JDK gets the same volume
78–84 and 120–124	`build_project()` loops over the JDK versions; each `docker run` mounts those same named volumes	Later JDK builds see whatever earlier builds downloaded
27–32 and 38	`remove_cache_volumes()` runs at startup and after each project finishes	Caches are cleared per project, not per JDK version

Key snippet (lines 119–124):

```
def build_project_with_java(project_dir, java_version, builder, log_dir):
    volumes = [f'--mount=type=volume,src={get_volume_name(cache)},dst={cache}' for
cache in CACHE_DIRS]
    command = ['docker', 'run', '--rm', '--quiet', f'--name={get_container_name()}',
               f'--
mount=type=bind,src={project_dir.resolve()},dst={DOCKER_PROJECT_SRC},readonly',
               *volumes,
               f'{IMAGE_NAME}:{java_version}', builder]
```

The `java_version` only changes the image tag; the cache volumes stay the same.

Options and decision

We made no change and kept the original study's shared-cache approach.

Option	Benefit	Cost
Separate cache per JDK version (e.g. add the JDK version to the volume name)	This would reduce the risk of one Java build affecting another.	Results no longer directly comparable with the original study; more downloads and run time
Keep the original approach (chosen)	Results stay comparable with the original study and our M2 work	Shared-cache validity threat remains

Rationale: our goal was to reproduce the original results, so changing the method would weaken the comparison. Testing different caching strategies is outside the focus of our research questions. We record it as a limitation and possible future work.

Trade-off: We have not tested whether sharing the cache changes build outcomes, so we cannot rule out that some passes or failures reflect cache state rather than the JDK version.